

AI Engineering Boot Camp

Day 10 — Your First Real ML Model

Concepts · Code · Results · What It All Means

Target: Google · Meta · OpenAI · Anthropic

The Big Picture — What We Built Today

Today everything from the past 9 days came together. We built a complete ML pipeline — from raw data to a trained model with visualised results. This is the exact workflow used at every ML company in the world.

File	What it does
day10_first_model.py	Complete ML pipeline: load, clean, split, scale, train, evaluate, visualise

The model we trained: Logistic Regression — a classifier that predicts whether someone will be hired based on their age, income, education, and experience. Final accuracy: 86%.

The 6-Step ML Workflow

Every ML project — from a simple classifier to a billion-parameter language model — follows this same workflow. Learn it once, apply it everywhere.

Step	What happens	Why it matters
1. Load & explore	Read data, check shape, types, missing values	You cannot build a good model on bad data
2. Clean & preprocess	Fix missing values, encode categories, scale features	Models break on messy data
3. Split data	Separate into train (80%) and test (20%) sets	Honest evaluation — test data never seen during training
4. Train model	<code>model.fit(X_train, y_train)</code>	Model learns patterns from training data
5. Evaluate	Accuracy, confusion matrix, classification report	Understand how and where the model fails
6. Visualise	Confusion matrix plot, feature importance	Charts make results understandable to everyone

Concept: X and y — Features and Target

Concept: X is the input matrix (capital — 2D). y is the target vector (lowercase — 1D). This notation is universal across all of ML.

Think of it like a spreadsheet. X is the left columns (inputs). y is the right column (what we predict):

age	income	education	experience	hired = y
56	49721	21	17	True
46	24897	12	4	False
32	33912	19	24	True

- X shape: (500, 4) — 500 rows, 4 features
- y shape: (500,) — 500 rows, 1 value each

```
# Separating X and y from the DataFrame
# .values converts DataFrame to plain NumPy array
X = df[['age', 'income', 'education', 'experience']].values
y = df['hired'].values
```

Concept: Train/Test Split — Honest Evaluation

Concept: Never evaluate a model on the data it trained on. That is like giving students the exam answers before the test — of course they score 100%.

- Training set (80%) — the model learns from this data
- Test set (20%) — the model never sees this during training. Used only for final evaluation.

Variable	Shape	Purpose
X_train	(400, 4)	80% of features — model trains on this
X_test	(100, 4)	20% of features — model evaluated on this
y_train	(400,)	80% of labels — model learns from these
y_test	(100,)	20% of labels — compared against predictions

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    # 20% goes to test set, 80% to training
    test_size=0.2,
```

```

# random_state=42 makes the split reproducible – same split every time
random_state=42
)

```

Concept: Feature Scaling — Equal Footing

Concept: StandardScaler transforms each feature to mean=0 and std=1. Without this, features with large number ranges dominate the model unfairly.

Feature	Raw range	After scaling
age	22 to 60	approx -1.5 to +1.5
income	20,000 to 100,000	approx -1.5 to +1.5
education	8 to 22	approx -1.5 to +1.5
experience	0 to 20	approx -1.5 to +1.5

```

scaler = StandardScaler()
# fit_transform: learn mean and std FROM training data, then scale
X_train_scaled = scaler.fit_transform(X_train)
# transform only: apply the SAME scale learned from training data
# NEVER call fit_transform on test data – that is data leakage
X_test_scaled = scaler.transform(X_test)

```

Warning: Data leakage: fitting the scaler on test data lets training use information from the test set. The model appears better than it actually is. Always fit on training data only.

Concept: Training a Model

Concept: The scikit-learn API is the same for every model: create, fit, predict. Three lines, same pattern always.

```

# Step 1: Create – configure the model, no learning yet
model = LogisticRegression(random_state=42)

# Step 2: Fit – model learns from training data
# Adjusts internal weights to minimise prediction errors
model.fit(X_train_scaled, y_train)

```

```
# Step 3: Predict – apply learned patterns to new unseen data
y_pred = model.predict(X_test_scaled)
```

Note: The same three lines work for ANY scikit-learn model. Swap `LogisticRegression()` for `RandomForestClassifier()` or `GradientBoostingClassifier()` — the pattern never changes.

Concept: What Is Logistic Regression?

Despite the name, logistic regression is a classifier — it predicts categories (hired/not hired), not continuous values. It learns one weight per feature during training.

The weights it learned for our data:

Feature	Coefficient	Meaning
experience	2.30	Strong positive — more experience means much more likely hired
education	0.81	Positive — more education means more likely hired
income	0.15	Weak — small effect on prediction
age	0.03	Near zero — age barely influences the model

These match exactly what we programmed — experience had weight 0.5, education had weight 0.3. The model successfully discovered the true signal in the data.

Concept: Evaluation Metrics

Accuracy

Concept: Accuracy = correct predictions / total predictions. Simple but limited.

Our model: 86% accuracy = 86 correct predictions out of 100 test samples.

Warning: Accuracy can mislead. If 95% of people are not hired, a model that always predicts not-hired gets 95% accuracy without learning anything. Always check the confusion matrix too.

Confusion Matrix

Concept: Shows exactly WHERE the model makes mistakes — not just how many.

	Predicted: NOT hired	Predicted: HIRED
Actual: NOT hired	True Negative (TN) = 49 Correct — both say not hired	False Positive (FP) = 9 Wrong — predicted hired, was not
Actual: HIRED	False Negative (FN) = 5 Wrong — predicted not hired, was hired	True Positive (TP) = 37 Correct — both say hired

The two correct cells (TN=49 and TP=37) sit on the diagonal. The two mistake cells (FP=9 and FN=5) sit off the diagonal. You want the diagonal numbers as large as possible.

Error type	What happened	Real cost
False Positive (9)	Said hired, was not hired	Wasted interview time
False Negative (5)	Said not hired, was hired	Lost a good candidate — often worse

Precision, Recall and F1

Metric	Formula	Question it answers
Precision	$TP / (TP + FP)$	Of everyone I predicted hired, how many actually were?
Recall	$TP / (TP + FN)$	Of everyone who actually was hired, how many did I find?
F1 Score	$2 \times (P \times R) / (P + R)$	Balance between precision and recall

Concept: Feature Importance

Concept: Feature importance shows which features the model relied on most. In logistic regression, larger coefficients mean more important features.

Why this matters in production:

- Features with near-zero importance can be dropped — simpler models generalise better
- Unexpected importance may indicate data leakage or a bug — always investigate
- Business teams need to understand why the model makes decisions to trust it
- Some industries require explainable models for regulatory compliance

```
# model.coef_[0] — array of weights learned, one per feature
```

```
coefficients = model.coef_[0]
feature_names = ['age', 'income', 'education', 'experience']
for name, coef in zip(feature_names, coefficients):
    print(f'{name}: {coef:.3f}')
```

Full Code Walkthrough — Line by Line

Every line explained. Green lines are comments explaining the code above or below them.

Section 1: Imports

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
# sklearn = scikit-learn. Each module has specific tools.
from sklearn.model_selection import train_test_split
# LogisticRegression: the classification model we are training
from sklearn.linear_model import LogisticRegression
# StandardScaler: scales all features to mean=0, std=1
from sklearn.preprocessing import StandardScaler
# Evaluation tools: accuracy, confusion matrix, full report
from sklearn.metrics import (accuracy_score, confusion_matrix, classification_report)
```

Section 2: Create Data With Real Signal

```
# experience and education are the features that actually predict hiring
# 0.5 * experience means experience has stronger effect than 0.3 * education
hired = (0.3 * education + 0.5 * experience
         + np.random.normal(0, 2, n) > 10).astype(int)
```

This creates realistic data where experience matters most. We will verify the model discovers this on its own.

Section 3: Features and Target

```
# X = features matrix. Capital X because it is 2D (rows x columns).
X = df[['age', 'income', 'education', 'experience']].values
# y = target vector. Lowercase because it is 1D (one value per row).
y = df['hired'].values
```

Section 4: Train/Test Split

```
# Split BEFORE any processing – test set must be untouched
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)
```

```
)
```

Result: 400 rows in X_train, 100 rows in X_test. The model only ever sees X_train during training.

Section 5: Scaling

```
scaler = StandardScaler()
# fit_transform: LEARN mean/std from training data, THEN apply scaling
X_train_scaled = scaler.fit_transform(X_train)
# transform ONLY: apply the SAME scale already learned from training data
# Do NOT fit on test data – that would be data leakage
X_test_scaled = scaler.transform(X_test)
```

Section 6: Train

```
# Create the model – no learning happens yet
model = LogisticRegression(random_state=42)
# fit: this is where learning happens
# Model adjusts weights to minimise prediction errors on training data
model.fit(X_train_scaled, y_train)
```

Section 7: Predict and Evaluate

```
# predict: apply learned weights to test data model has never seen
y_pred = model.predict(X_test_scaled)
# accuracy_score: correct predictions / total predictions
accuracy = accuracy_score(y_test, y_pred)
# classification_report: precision, recall, F1 for each class
print(classification_report(y_test, y_pred,
                           target_names=['not hired', 'hired']))
```

Section 8: Feature Importance

```
# model.coef_[0]: weights the model assigned to each feature
# Larger = more important. [0] because sklearn stores it in 2D array.
coefficients = model.coef_[0]
for name, coef in zip(feature_names, coefficients):
    print(f'{name}: {coef:.3f}')
```

Results — What the Model Found

Accuracy: 86%

86 out of 100 test predictions were correct. Strong result for a first model on noisy data.

Confusion Matrix

	Predicted: not hired	Predicted: hired
Actual: not hired	49 correct (TN)	9 wrong (FP)
Actual: hired	5 wrong (FN)	37 correct (TP)

14 total mistakes: 9 false positives and 5 false negatives. Missing a good candidate (FN) is typically more costly than a wasted interview (FP).

Feature Importance

Feature	Coefficient	Verdict
experience	2.30	Most important — model learned this correctly
education	0.81	Second — model learned this correctly
income	0.15	Weak signal — correctly identified as low importance
age	0.03	No signal — correctly identified as irrelevant

Anki Cards — Day 10

- Q: What is X in ML notation? A: The feature matrix — 2D, capital letter, shape (samples, features)
- Q: What is y in ML notation? A: The target vector — 1D, lowercase, shape (samples,)
- Q: Why split into train and test sets? A: Honest evaluation — never evaluate on data the model trained on
- Q: What is data leakage? A: Using test data during training — makes model appear better than it is
- Q: What does StandardScaler do? A: Transforms features to mean=0, std=1 — puts all features on equal footing
- Q: Why fit scaler on training data only? A: Fitting on test data leaks information — always fit on train only
- Q: What are the 3 steps of the sklearn API? A: create, `model.fit(X_train, y_train)`, `model.predict(X_test)`
- Q: What is a False Positive? A: Predicted positive (hired) but actually negative (not hired)
- Q: What is a False Negative? A: Predicted negative (not hired) but actually positive (hired)
- Q: What does feature importance tell you? A: Which features the model relies on most

Next: [Day 11 — Random Forests and model comparison.](#)